Two implementations of the same MPM

`default` against `warp`: where the time goes, and why

1. The result

The four MLS-MPM operators — `mpm_strain`, `mpm_scatter`, `mpm_grid_update`, `mpm_gather` — exist in two forms a spec picks by name on the `implementation` axis. `default` is pure PyTorch: 2D and 3D, CPU or CUDA, and differentiable in use, not merely in principle (`engine.run(..., grad=True)` keeps the tape across a rollout and `discovery_cardio_mpm/train.py` backpropagates through exactly these four). `warp` is one NVIDIA Warp kernel per operator, 3D and CUDA only. Same equations, so they sit on the `implementation` axis and not the `model` axis: swapping one for the other asserts the physics did not change, and §6 tests that assertion.

On `si_waterfall` — 570 760 particles, 96^3 grid, 13 substeps, A100: 973.8 ms/frame for `default` against **31.8 ms/frame** for `warp` with the grid solve fused and the substep captured — 30.6×. At 5 M particles on a 192^3 grid it is 16 965 against 441 ms, 38.5×. One RTX A6000 runs 150 M particles; one B300 runs **one billion**.

The mechanism is traffic, not memory held. Traffic here means bytes moved between the GPU's DRAM and its compute units — read into the streaming multiprocessors and written back out. It is the quantity a memory-bound kernel actually spends its time on. Every intermediate tensor an operator allocates is one DRAM write plus at least one DRAM read by whatever consumes it; a value held in a register never leaves the chip and costs none. `default`'s gather materialises two $[N, 27, 3, 3]$ tensors — 918.5 MB each at 945 k particles — to produce a $[N, 3, 3]$ result of 34.0 MB. The Warp kernel accumulates the same sum in nine registers and writes none of it.

It is tempting to read the peak-memory drop (2.94 GiB → 0.35 GiB) as the cause. **Peak memory is a stock and traffic is a flow**, and §5.2 separates them: on one substep of the gather at 570 760 particles, the batched form allocates **3511.9 MB** and the loop form **1924.7 MB** — 1.82× less traffic, 1.25× less peak, and 3.70× less time. Time tracks the flow, not the stock.

2. The cycle

MLS-MPM carries material on particles p and solves on a background grid i thrown away every substep. With Δx the cell size, w_{ip} the quadratic B-spline weight of node i at particle p, and $\mathbf{x}_{ip} = \mathbf{x}_i - \mathbf{x}_p$:

$$\textbf{strain} \quad \mathbf{F}_p \leftarrow (\mathbf{I} + \Delta t\, \mathbf{C}_p)\, \mathbf{F}_p \tag{1}$$

$$\textbf{scatter} \quad m_i = \sum_p w_{ip} m_p, \qquad (m\mathbf{v})_i = \sum_p w_{ip}\big[m_p \mathbf{v}_p + \mathbf{A}_p\, \mathbf{x}_{ip}\big] \tag{2}$$

$$\mathbf{A}_p = -\tfrac{4\Delta t}{\Delta x^2} V_p^0\, \boldsymbol{\sigma}_p + m_p \mathbf{C}_p, \qquad \boldsymbol{\sigma}_p = 2\mu(\mathbf{F}_p - \mathbf{R}_p)\mathbf{F}_p^\top + \lambda J(J-1)\mathbf{I} \tag{3}$$

$$\textbf{grid} \quad \mathbf{v}_i = (m\mathbf{v})_i / \max(m_i, \epsilon), \quad \text{then gravity, walls, plates, surface tension} \tag{4}$$

$$\textbf{gather} \quad \mathbf{v}_p = \sum_i w_{ip} \mathbf{v}_i, \qquad \mathbf{C}_p = \tfrac{4}{\Delta x} \sum_i w_{ip}\, \mathbf{v}_i \otimes (\mathbf{o}_i - \mathbf{f}_p), \qquad \mathbf{x}_p \leftarrow \mathbf{x}_p + \Delta t\, \mathbf{v}_p \tag{5}$$

$\mathbf{F}_p$ is the deformation gradient, one 3×3 per particle: the Jacobian of the map from a particle's reference position to its current one, carrying accumulated stretch, shear and rotation since $t = 0$. $\mathbf{R}_p$ is its rotation part and $J = \det \mathbf{F}_p$ the volume ratio. $\mathbf{o}_i - \mathbf{f}_p$ is $\mathbf{x}_{ip}$ in cell units. **The stencil is $3^3 = 27$ nodes**, because a quadratic B-spline spans three cells per axis and three dimensions multiply. Every per-particle cost below carries that 27.

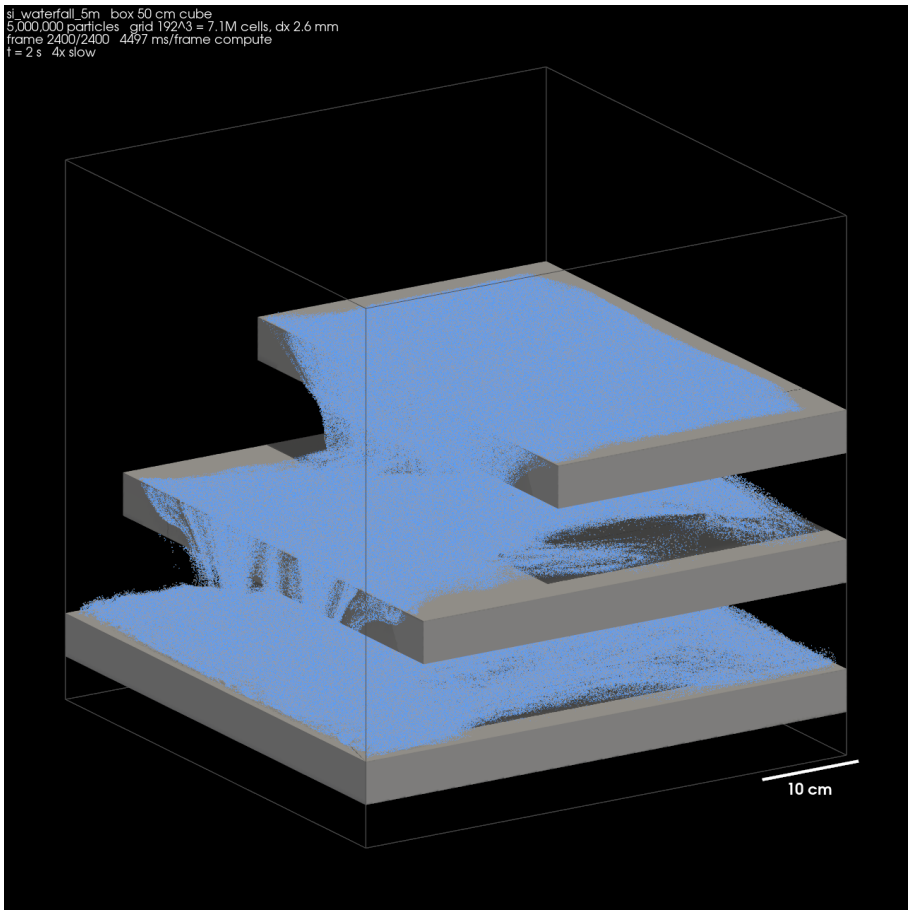

Figure 1: `si_waterfall_5m`: 5 000 000 particles of water in a 50 cm cube on a 192^3 grid ($\Delta x = 2.6$ mm), 26 substeps per frame, at $t = 2$ s. Every particle is simulated; the picture is taken off the GPU as the run proceeds. This spec is one of the two the sweep in §7 uses, and it is dimensional — the box, the scale bar and Δx are in real units, so `4497 ms/frame` buys 2 seconds of water rather than 2 seconds of nothing in particular.

The contract both sign. `mpm_scatter` reads `pos, vel, C, F, mass, mu, la, pvol` and the parent's acceleration, and accumulates into `mpm_grid.m` and `.mv` in place. It must **not** touch `F`: an early attempt fused the strain update into the scatter for free and advanced `F` twice per substep — grid mass still agreed to eleven digits while `F` disagreed at 3.3×10^{-4}. `mpm_gather` reads `mpm_grid.v`, writes `pos`, `vel`, `C` in place, and leaves dormant particles where they are.

3. Why `default` is slow

The gather is the shorter operator and makes the point completely. N is the particle count, $S = 27$, $D = 3$; sizes are at $N = 945\,000$.

```
fx, weight, flat = bspline(X, inv_dx, offsets, g.shape, periodic)
    gpos = base[:,None,:] + oidx[None] # [N,27,3] i64 612.4 MB
    flat = ((c0 * ny) + c1) * nz + c2 # [N,27] i64 204.1 MB x4
gvn = g.v[flat].view(p.n, S, D) # [N,27,3] f32 306.2 MB
new_V = (weight[..., None] * gvn).sum(1) # [N,27,3] f32 306.2 MB
dpos_grid = offsets[None] - fx[:, None, :] # [N,27,3] f32 306.2 MB
new_C = 4*inv_dx*(weight[...,None,None] *
       (gvn[...,:,None] @ dpos_grid[...,None,:])).sum(1) # [N,27,3,3] f32 918.5 MB x2
```

The last line is the whole story. That expression is $N \times 27$ independent $3{\times}1$ by $1{\times}3$ outer products, and PyTorch has no way to say "form this $3{\times}3$, weight it, add it to a running sum, discard it". It must materialise all $N \times 27$: write **918.5 MB**, read it back to multiply by `weight`, write **another 918.5 MB**, read that back to reduce over the stencil axis, and only then produce the $[N, 3, 3]$ anyone

wanted — **34.0 MB**. **The useful output is 1.8% of the traffic spent producing it.**

3.1 The full bill, measured

The listing is read off the source; the tables are measured. A `TorchDispatchMode` records every `aten` call one real substep makes and the shape and dtype of everything it allocates — including what the source never names: broadcast temporaries, the `index` gathers hiding behind `w[:, oidx[:,k], k]`, the int64 index arithmetic. Views are excluded, since they move nothing and counting them would inflate `default`'s bill with bytes it never touches. Rows below 40 MB are omitted; totals include everything.

`mpm_gather, default` ($N = 945\,000$, one substep)

aten op	shape	dtype	each	×	total
`bmm`	`[25515000, 3, 3]`	float32	918.5 MB	1	918.5 MB
`mul`	`[945000, 27, 3, 3]`	float32	918.5 MB	1	918.5 MB
`add`	`[945000, 27, 3]`	int64	612.4 MB	1	612.4 MB
`clamp`	`[945000, 27]`	int64	204.1 MB	3	612.4 MB
`mul`	`[945000, 27]`	int64	204.1 MB	2	408.2 MB
`add`	`[945000, 27]`	int64	204.1 MB	2	408.2 MB
`index`	`[945000, 27]`	float32	102.1 MB	3	306.2 MB
`mul`	`[945000, 27]`	float32	102.1 MB	3	306.2 MB
`index`	`[25515000, 3]`	float32	306.2 MB	1	306.2 MB
`mul`	`[945000, 27, 3]`	float32	306.2 MB	1	306.2 MB
`sub`	`[945000, 27, 3]`	float32	306.2 MB	1	306.2 MB
`ones`	`[945000, 27]`	float32	102.1 MB	1	102.1 MB
`mul`	`[945000, 3]`	float32	11.3 MB	6	68.0 MB
`sub`	`[945000, 3]`	float32	11.3 MB	4	45.4 MB
all allocating ops					**6060 MB**
per particle					**6413 B**
÷ essential (864 B)					**7.4**×

`mpm_scatter, default` (same substep)

aten op	shape	dtype	each	×	total
`clone`	`[945000, 27, 3, 3]`	float32	918.5 MB	1	918.5 MB
`mul`	`[945000, 27]`	float32	102.1 MB	6	612.4 MB
`add`	`[945000, 27, 3]`	int64	612.4 MB	1	612.4 MB
`clamp`	`[945000, 27]`	int64	204.1 MB	3	612.4 MB
`mul`	`[945000, 27, 3]`	float32	306.2 MB	2	612.4 MB
`mul`	`[945000, 27]`	int64	204.1 MB	2	408.2 MB
`add`	`[945000, 27]`	int64	204.1 MB	2	408.2 MB
`index`	`[945000, 27]`	float32	102.1 MB	3	306.2 MB
`sub`	`[945000, 27, 3]`	float32	306.2 MB	1	306.2 MB
`bmm`	`[25515000, 3, 1]`	float32	306.2 MB	1	306.2 MB
`add`	`[945000, 27, 3]`	float32	306.2 MB	1	306.2 MB
`index_select`	`[945000, 3, 3]`	float32	34.0 MB	8	272.2 MB
`mul`	`[945000, 3, 3]`	float32	34.0 MB	8	272.2 MB
`add`	`[945000, 3, 3]`	float32	34.0 MB	6	204.1 MB
`linalg_cross`	`[945000, 3, 3]`	float32	34.0 MB	4	136.1 MB
`div`	`[945000, 3, 3]`	float32	34.0 MB	4	136.1 MB
`mul`	`[945000, 3]`	float32	11.3 MB	10	113.4 MB
`ones`	`[945000, 27]`	float32	102.1 MB	1	102.1 MB
`sub`	`[945000, 3]`	float32	11.3 MB	5	56.7 MB
all allocating ops					**7177 MB**
per particle					**7594 B**
÷ essential (864 B)					**8.8**×

Per particle per substep the four `default` operators allocate **313 B** + **7594 B** + **151 B** + **6413 B**. The *essential* traffic — what any correct MLS-MPM must move — is 216 floats = 864 B: read `pos/vel/C/F/mass/mu/la`, scatter mass and momentum to 27 nodes, gather velocity back from 27. `default` moves **16.7**× the algorithm's requirement, and the two $[N, 27, 3, 3]$ tensors are most of it.

4. Why `warp` is fast

One thread per particle. Everything the tables list as a tensor is a local variable, and a local variable in a CUDA thread is a register, which is not memory.

Nothing is ever 27 wide. A thread has ≈64 registers — nowhere near enough for 27 copies of (weight, node velocity, offset). The loop *visits* the 27 nodes one at a time and accumulates. Only `newv` (3 floats) and `newC` (9 floats) persist across iterations; each node's weight, velocity and offset live for one iteration. The register cost is $O(1)$ in the stencil size, not $O(27)$ — exactly what PyTorch cannot express, having no way to form a value, add it to a running sum and discard it, and so making the loop a tensor axis instead.

`ptxas -arch=sm_86 -O3 -verbose` confirms the arithmetic fits: **64 registers, 0 bytes stack frame, 0 spill stores, 0 spill loads** for both `g2p` and `p2g_atomic`. **Zero spill is the load-bearing number**: had the working set exceeded the register file, the compiler would have pushed the overflow to local memory — global memory under another name — and the fusion would have bought nothing.

```
@wp.kernel # src/plexus/operators/mpm_warp.py:193
def g2p(X, STATE, C, GV, OCC, LIQ, pa, va, ngx, ngy, ngz, dx, dt, ...):
    p = wp.tid() # one thread per particle
    newv = wp.vec3(0,0,0) # vec3 -> 3 registers
    newC = wp.mat33(0,...,0) # mat33 -> 9 registers
    for i in range(3): # the 27-node stencil, UNROLLED by the compiler
      for j in range(3):
        for k in range(3):
          w = wi*wj*wk # the [N,27] 'weight' tensor: 1 register
          gv = GV[(gi*ngy + gj)*ngz + gk] # the [N,27,3] 'gvn' tensor: 3 registers
          dpos = wp.vec3(i-fx[0], j-fx[1], k-fx[2]) # the [N,27,3] 'dpos_grid': 3 registers
          newv = newv + w*gv # the [N,27,3] product: never exists
          newC = newC + (4*inv_dx*w)*wp.outer(gv, dpos) # the [N,27,3,3]: NEVER EXISTS
    STATE[p, pa+0..2] = xn ; STATE[p, va+0..2] = newv ; C[p] = newC # the only writes
```

quantity	`default` tensor	at $N = 945\,000$	`warp`
B-spline weights per axis	`[N,3,3]` f32	34.0 MB	3 registers, recomputed per tap
stencil weight	`[N,27]` f32	102.1 MB ×6	1 register
node indices	`[N,27,3]` i64	612.4 MB	3 integers, computed inline
flattened index	`[N,27]` i64	204.1 MB ×4	1 integer
gathered node velocity	`[N,27,3]` f32	306.2 MB	`wp.vec3`, 3 registers
$\mathbf{x}_{ip}$ in cells	`[N,27,3]` f32	306.2 MB	`wp.vec3`, 3 registers
outer product $\mathbf{v}_i \otimes \mathbf{x}_{ip}$	`[N,27,3,3]` f32	**918.5 MB** ×2	**accumulated into 9 registers**
allocated per substep, both fused operators		**13 237 MB**	**0 MB**

The traced run confirms it: `mpm_scatter` and `mpm_gather` do not appear in the allocating-op trace at all.

The scatter differs from the gather in one way. Grid→particle is a pure read: no two threads write the same place, so there is nothing to coordinate. Particle→grid is the opposite — many particles land on one node — and the kernel uses `wp.atomic_add`, 108 per particle (27 nodes × (1 mass + 3 momentum)). That is why the two fusions do not cost the same, and it is the remaining wall (§8).

Differentiability is unwired, not lost. Warp emits a backward kernel beside every forward one, so `DIFFERENTIABLE = False` is not a statement about the tool. This port launches with a plain `wp.launch` outside a `wp.Tape`, and `wp.from_torch` wraps tensors without `requires_grad`, so no backward is registered with autograd. Until that is wired, anything needing a gradient stays on `default` — which, with byte-identity (§6), is why `default` is not going away.

5. What it buys

5.1 Which fusion did what

Each operator switched on its own, 945 000 particles, RTX A6000, 12 timed frames after 4 warm-up.

`mpm_scatter`	`mpm_gather`	ms/frame	peak	speedup
default	default	1150.8	2.94 GiB	1.0×
warp	default	466.9	2.80 GiB	2.5×
default	**warp**	767.8	2.94 GiB	1.5×
warp	**warp**	79.4	0.35 GiB	14.5×
warp	**warp** +capture	52.2	0.55 GiB	22.0×

The two fusions are superlinear together. Alone they are worth 2.5× and 1.5×; together 14.5×, and $2.5 \times 1.5 = 3.75$. Fusing one end leaves the other end's 918.5 MB temporary alive, so the allocator's high-water mark does not move and every other kernel in the substep keeps paying for it. The last 918.5 MB is not one seventh of the problem — it is the problem.

5.2 The control: removing the temporaries *without* leaving PyTorch

`mpm_gather[torch_loop27]` is a third implementation written to separate two explanations that the ablation above cannot: is `warp` fast because it moves fewer bytes, or because it holds less memory? It walks the 27 stencil offsets in a Python loop and accumulates $[N, D]$ — so it has `warp`'s traffic profile and `default`'s everything else.

And traffic is measurable, so it need not be inferred from the memory proxy. The same `TorchDispatchMode` that produces the tables in §3.1 sums the bytes of every tensor an `aten` call allocates over one substep — which is the DRAM traffic, up to the reads that consume each write. On the gather at $N = 570\,760$:

`mpm_gather`	`aten` allocs	allocated	B/particle	ms/call
`default` (batched)	79	**3511.9 MB**	6452	**34.20**
`torch_loop27`	292	**1924.7 MB**	3536	**9.24**

1.82× **less traffic,** 3.70× **less time**, against 1.25× less peak memory (1.74 → 1.39 GiB). All three move the same way, but only traffic moves far enough to account for the speed: **peak allocation is the weakest of the three predictors and the one the headline used to quote.** The speedup exceeding the traffic ratio is the caching allocator and the L2 — a 918.5 MB temporary is evicted before anything reads it, a $[N, 3]$ one is not — so 1.82 is a lower bound on what removing the tensors was worth, not a proportionality constant.

`torch_loop27` also stays 12–27× slower than `warp` (§7), which prices what a Python loop cannot recover: 292 kernel launches where the Warp kernel is one, and no registers. `warp` wins on traffic *and* on launches, and this is the measurement that says so rather than assuming it.

5.3 Scaling

RTX A6000 (48 GiB, peak 768 GB/s), `warp`, capture off:

particles	sub	ms/frame	ms/substep	GB/s	% peak
945 000	7	77.8	11.12	73.4	9.6%
1 999 998	7	161.8	23.12	74.7	9.7%
4 999 995	7	402.1	57.44	75.2	9.8%
9 999 990	7	784.9	112.13	77.1	10.0%
20 000 007	7	1614.1	230.58	74.9	9.8%
100 000 008	7	7968.9	1138.42	75.9	9.9%

Throughput is **flat at ≈75 GB/s across a 106× range** — 73.4 GB/s at 945 k, 75.9 at 100 M — and memory linear at 0.309 GiB per million particles. No knee anywhere. On the A100, `default` at **10 M** needed **30.5 GiB** and **11.60 s/frame**; `warp` fits **ten times the particles in the same memory**, and a 100 M frame costs *less* than a 10 M `default` frame did.

	capture off			**capture on**		
particles	ms/frame	GB/s	GiB	ms/frame	GB/s	GiB
500 013	40.7	74.2	0.22	**24.0**	126.2	0.37
999 999	71.3	84.9	0.37	**46.7**	129.6	0.58
4 999 995	378.5	79.9	1.61	**229.8**	131.6	2.29
9 999 990	778.6	77.7	3.14	**459.5**	131.6	4.42
20 000 007	1544.8	78.3	6.23	**913.2**	132.5	8.68
100 000 008	7853.9	77.0	30.89	**4551.0**	132.9	42.81

particles	sub	ms/frame	ms/substep	GB/s	% peak
500 013	7	581.3	83.05	5.2	0.3%
999 999	7	1158.3	165.47	5.2	0.3%
4 999 995	7	5790.6	827.23	5.2	0.3%
9 999 990	7	11598.0	1656.86	5.2	0.3%

`default` **does not reach 20 M on an 80 GiB card.** It dies asking for **18.11 GiB in a single allocation** at 20 M and **60.35 GiB** at 100 M — individual temporaries, not totals.

The most informative row in this note is a `default` row. On the A100 `default` runs at **5.2 GB/s**; on the A6000, **5.0**. Twice the memory bandwidth buys the prior implementation **4%**. It was never bandwidth-bound — it was bound by kernel-launch count and allocator work, neither of which a faster card improves. **And `warp` is not bandwidth-bound either**: at 100 M it runs 7853.9 ms on the A100 against 7968.9 on the A6000, 1.5% for twice the bandwidth. Both land at ≈77 GB/s — 5% of A100 peak, 10% of A6000 peak. The *same absolute* throughput on very different hardware is the signature of a serialised bottleneck, which is the scatter's atomics.

5.4 CUDA-graph capture

`capture: true` records a substep block once as a `torch.cuda.CUDAGraph` and replays it, instead of re-issuing kernels through Python. It is orthogonal to `implementation`.

What it removes is not launch latency. It was worth 1.73× at 100 M (7853.9 → 4551.0 ms) — but at 4.5 s a frame, 28 elided launches are nanoseconds against seconds. The memory column names the real mechanism: peak *rises* 30.89 → 42.81 GiB, because a captured graph allocates from a private pool that stays resident, so the un-fused operators' $[N, 3, 3]$ intermediates are allocated once instead of seven times a frame. Capture was buying the removal of the caching allocator — the same thing fusion buys, for the operators fusion had not reached.

That prediction has now been tested and held. With `mpm_grid_update` ported (§7), capture is worth 1.02–1.09× (§7.2). It was a stand-in for un-fused operators, and as they are fused it evaporates while its 39% memory cost remains. It should default to off once `mpm_strain` lands.

It fails silently, which is its real cost. A capture pool that does not fit does not raise; the allocator retries, so the run sits at 100% CPU with no output, no error and no progress, indefinitely. The diagnostic is an *absence* — the engine prints `substep captured as a CUDA graph` on success, and that line never appears. `Plexus_Main.py` now projects the footprint before the run (0.42 GiB per million captured against 0.309 eager), compares it with *that card's* memory, and warns past 80%. A fixed threshold would nag on an 80 GiB H100 and stay silent on a 24 GiB card.

6. Precision

`warp` is not byte-identical to `default` and cannot be: the 27-tap sums are reassociated (sequential in the kernel, pairwise in `.sum(1)`), and `wp.atomic_add` commits in whatever order the hardware

schedules. So this is an `implementation` with a tolerance gate, not a promotion twin. Measured after 2 frames × 7 substeps on 108 k particles from one seed:

switched	quantity	max $\lvert\Delta\rvert$	max relative	reference scale
gather only	`pos`	1.79×10^{-7}	4.3×10^{-7}	$\lvert x\rvert \approx 0.40$
gather only	`F`	8.34×10^{-7}	8.3×10^{-7}	$\lvert F\rvert \approx 0.33$
both	`pos`	1.79×10^{-7}	5.3×10^{-7}	$\lvert x\rvert \approx 0.40$
both	grid `mv`	1.46×10^{-11}	—	$\lvert mv\rvert \approx 1.8 \times 10^{-8}$

Float32 has $\epsilon = 1.19 \times 10^{-7}$, so a position of 0.40 has an ulp of 2.98×10^{-8} and the observed 1.79×10^{-7} is **6 ulp after 14 substeps** — round-off at the rate reassociation predicts. The large *relative* figures on the grid are on nodes whose mass is $\approx 10^{-8}$: a tiny denominator, not a disagreement. Total grid mass matches to six digits.

A run therefore cannot be reproduced bit-for-bit across the two, so `implementation` is part of a run's identity and must be recorded with it. Anything needing byte-identity stays on `default`.

7. The sweep, one year on

Everything above compares one scene at a fixed 96^3 grid. Two things have changed: `mpm_grid_update` now has a `warp` implementation, and a defect outside the MPM operators entirely was costing 4.4× on every run in the repository.

7.1 The aggregate, and why it dominated everything

`engine.py` used to set `torch.use_deterministic_algorithms(True)` at import, so two runs of a spec would agree bit for bit. That flag reroutes CUDA `index_add_` from atomics to a sort-and-segment kernel whose cost is driven by the longest run of *duplicate* indices — and `aggregate`, reducing 570 760 particles into a set with a *single* parent, hands it 570 760 copies of the index `0`. On an RTX A6000, `index_add_` of $[570760, 3]$ into $[1, 3]$: **140.98 ms** deterministic, 1.00 ms with atomics, **0.016 ms** for `sum(0)`.

Those two calls were **214 ms of a 255 ms frame — 84% of the simulation spent taking the centroid of a set with one member**, while the four MPM operators together took 41 ms. With one parent the index carries no information, so the segment sum *is* the sum. `sum(0)` is also the more accurate of the two: a tree reduction's error grows with $\log N$ where the sequential scan's grows with N, and against a float64 evaluation of the same sum (2.75×10^5) the errors are 2.1×10^{-2} and 5.9×10^2. Determinism is now opt-in through `PLEXUS_STRICT_DETERMINISM=1`, which the promotion gate already exported; no spec in the corpus of 2 456 ever asked for it.

7.2 Which implementation, two scenes, two cards

scene	implementation	ms/frame	ms/substep	ns/p-substep	peak GiB	capture
si_waterfall, 570,760 particles, 13 substeps/frame — NVIDIA A100-SXM4-80GB						
	`warpgrid_cap`	31.8	2.45	4.29	0.32	yes
	`warpgrid_eager`	32.4	2.49	4.37	0.22	–
	`warp_cap`	38.7	2.98	5.22	0.34	yes
	`warp_eager`	41.6	3.20	5.61	0.24	–
	`torch_loop27_cap`	434.0	33.38	58.49	1.55	yes
	`torch_loop27_eager`	450.1	34.62	60.66	1.39	–
	`torch_cap`	958.9	73.76	129.23	1.91	yes
	`torch_eager`	973.8	74.90	131.24	1.74	–
si_waterfall, 570,760 particles, 13 substeps/frame — NVIDIA RTX A6000						
	`warpgrid_cap`	35.0	2.69	4.72	0.32	yes
	`warpgrid_eager`	35.3	2.71	4.75	0.22	–
	`warp_cap`	52.7	4.05	7.10	0.34	yes
	`warp_eager`	55.1	4.24	7.42	0.24	–
	`torch_loop27_cap`	627.8	48.29	84.61	1.55	yes
	`torch_loop27_eager`	640.4	49.26	86.31	1.39	–
	`torch_cap`	952.2	73.25	128.33	1.91	yes
	`torch_eager`	962.9	74.07	129.77	1.74	–
si_waterfall_5m, 5,000,000 particles, 26 substeps/frame — NVIDIA A100-SXM4-80GB						
	`warpgrid_cap`	441.0	16.96	3.39	2.71	yes
	`warpgrid_eager`	442.1	17.00	3.40	1.90	–
	`warp_cap`	590.5	22.71	4.54	2.81	yes
	`warp_eager`	601.4	23.13	4.63	2.01	–
	`torch_loop27_cap`	7622.6	293.18	58.63	12.75	yes
	`torch_loop27_eager`	7696.2	296.01	59.20	11.88	–
	`torch_cap`	16839.3	647.67	129.53	15.91	yes
	`torch_eager`	16965.5	652.52	130.50	14.93	–
si_waterfall_5m, 5,000,000 particles, 26 substeps/frame — NVIDIA RTX A6000						
	`warpgrid_cap`	933.0	35.88	7.18	2.71	yes
	`warpgrid_eager`	934.6	35.95	7.19	1.90	–
	`warp_cap`	1267.5	48.75	9.75	2.81	yes
	`warp_eager`	1272.6	48.95	9.79	2.01	–
	`torch_loop27_cap`	10913.0	419.73	83.95	12.75	yes
	`torch_loop27_eager`	10974.1	422.08	84.42	11.88	–
	`torch_cap`	16668.8	641.11	128.22	15.91	yes
	`torch_eager`	16767.7	644.91	128.98	14.93	–

`warp` is 25–32× `default`. The fused grid solve adds 1.22× on the A100 and 1.51× on the A6000 — *more on the weaker card*, because the A100 gets through the torch grid update fast enough that removing it matters less. Capture is worth only 1.02–1.09×, against 1.73× before the port, exactly as §5.4 predicted.

7.3 How far one GPU goes

GPU	particles	n_{grid}	substeps	ms/frame	ns/p-substep	peak GiB
H100 80GB HBM3	200 M	192	13	5410	2.08	51.7
B300 SXM6 AC	800 M	296	20	14127	0.88	206.8
B300 SXM6 AC	1000 M	320	22	18918	0.86	258.4

The marginal cost is **276 B per particle**, measured two-point, so an 80 GiB H100 holds ≈290 M and a 267.7 GiB B300 ≈970 M. **One billion particles fits** with about 9 GiB to spare, and only with `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` — without it the run dies asking for the 33.5 GiB `F` buffer against 13.6 GiB free and 5.6 lost to fragmentation. **Throughput improves with size**: 2.08 ns per particle-substep at 200 M against 0.86 at 1 B, because a bigger problem hides the launch and tail effects a smaller one cannot.

8. What is left

1. `mpm_strain` **is the only** `default` **operator left** — 313 B per particle-substep against `warp`'s zero for the other three. It is a $[N, 3, 3]$ matrix product and should fuse trivially. `mpm_grid_update` was ported and moved from 42.7% of the frame to 11.2% (1.860 → 0.328 ms per call, 5.67×).

2. **The scatter is now 58.6% of the frame, and the wall is atomics.** `warp` sits at 8–10% of peak on both cards and does not improve when the card gets faster — the same signature `default` showed for a different reason. The fix is the node-centric P2G that NVIDIA's own `warp.fem` MPM uses: sort particles by cell into CSR, one thread *per grid node*, each summing its own contributions with a plain `+=`, zero atomics. The 27× read amplification that argument was once rejected on is a coalesced streaming read, which is not the same kind of cost as a serialised atomic write.

3. **Resolution is the constraint now, not memory.** A billion particles run on one B300, so the question is no longer how many fit but what grid they need. A finer grid costs substeps through the CFL bound $\Delta t \leq \mathrm{cfl} \cdot \Delta x / c$: the 1 B run needs 320^3 and 22 substeps where 200 M needs 192^3 and 13, so cost grows faster than particle count. That is a different scaling problem from this one and the next thing worth measuring.

4. **Half precision does not help and was not kept.** `torch.autocast(float16)` around the substep, with fp32 state and integration, made `warp` slower (34.4 → 37.3 ms/frame) and `torch` *much* slower (952.6 → 1772.4) — cast traffic on every intermediate, and it cannot see Warp kernels at all, since autocast rewrites `aten` ops and Warp compiles its own types. `torch.linalg.det` has no half kernel either.

Provenance. Benchmarks: `tools/mpm_bench.py` (bandwidth sweep) and `tools/mpm_perf.py` (implementation sweep, §7); tables regenerated by `tools/mpm_warp_note.py` and `tools/mpm_perf.py -latex`. Gates: `tools/mpm_impl_gate.py`. Specs: `material_3d_water_bench_500k` through `_200m` for §3–§6 — 27 blobs in a unit box, 96^3 grid, $\Delta t = 3.6 \times 10^{-3}$ in 7 substeps, held fixed so the only variable is particle count — and `si_waterfall`, `si_waterfall_5m`, `si_bench_200m/800m/1b` for §7. A production run would use a finer grid and more substeps; these sweeps deliberately do not, because a benchmark that changes two things at once measures neither.